SWIPE TO LEARN

Context Window vs. Fine-Tuning vs. RAG

The Decision Matrix for Building AI Applications

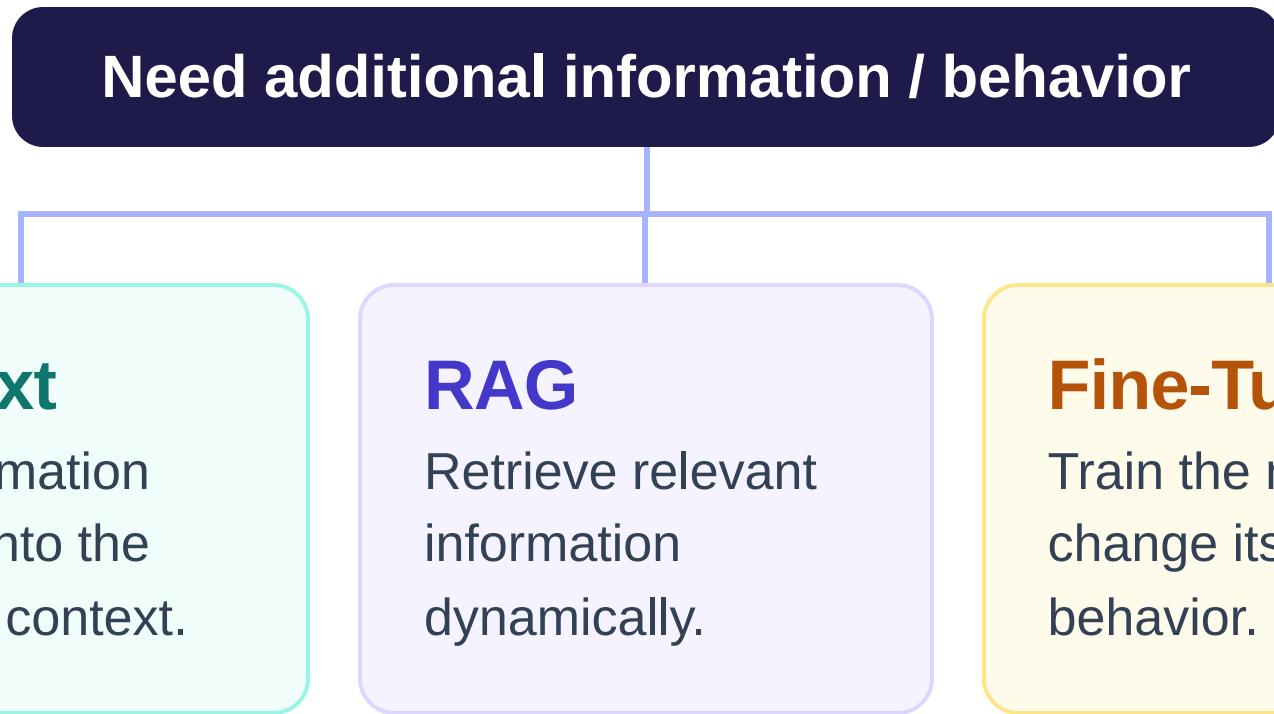
How to choose the right strategy for knowledge, behavior, and context

LONG CONTEXT | RAG | FINE-TUNING

Knowledge → Context → Behavior

Where Should the Model Get the Information?

Your application needs the model to answer using additional information. **What should you do?**



The right choice depends on whether you need more context, dynamic knowledge, or changed model behavior.

Knowledge vs. Behavior

Knowledge

"What does the model need to know?"

- Company policies
- Product documentation
- Customer records
- Current pricing
- Internal database information

Need more INFORMATION?

↓

Context **RAG**

Behavior

"How should the model respond?"

- Output format
- Tone
- Style
- Specialized syntax
- Consistent classification behavior

Need different BEHAVIOR?

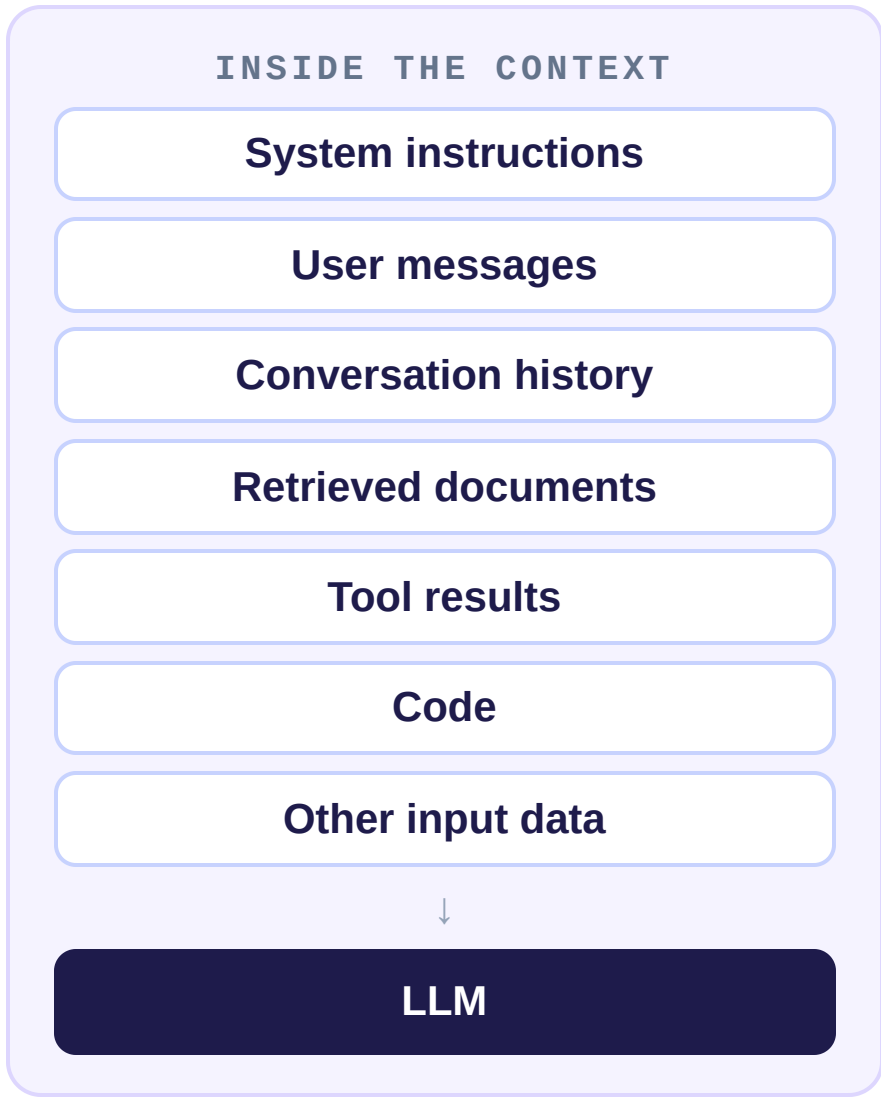
↓

Fine-Tuning

Important: Fine-tuning can encode information, but it is generally not the preferred mechanism for frequently changing factual knowledge.

Context Window = What the Model Can See

- The amount of information a model can process as part of a **single interaction**.
- It is **finite** — measured in tokens.
- Everything in it is supplied at **inference time**.



Context is provided at inference time; it does not permanently modify the model.

When Long Context Makes Sense

Best when you need to analyze a **relatively small number of large documents or inputs** immediately.



One large contract

Compare two technical docs

Codebase snapshot

Long report

Meeting transcript

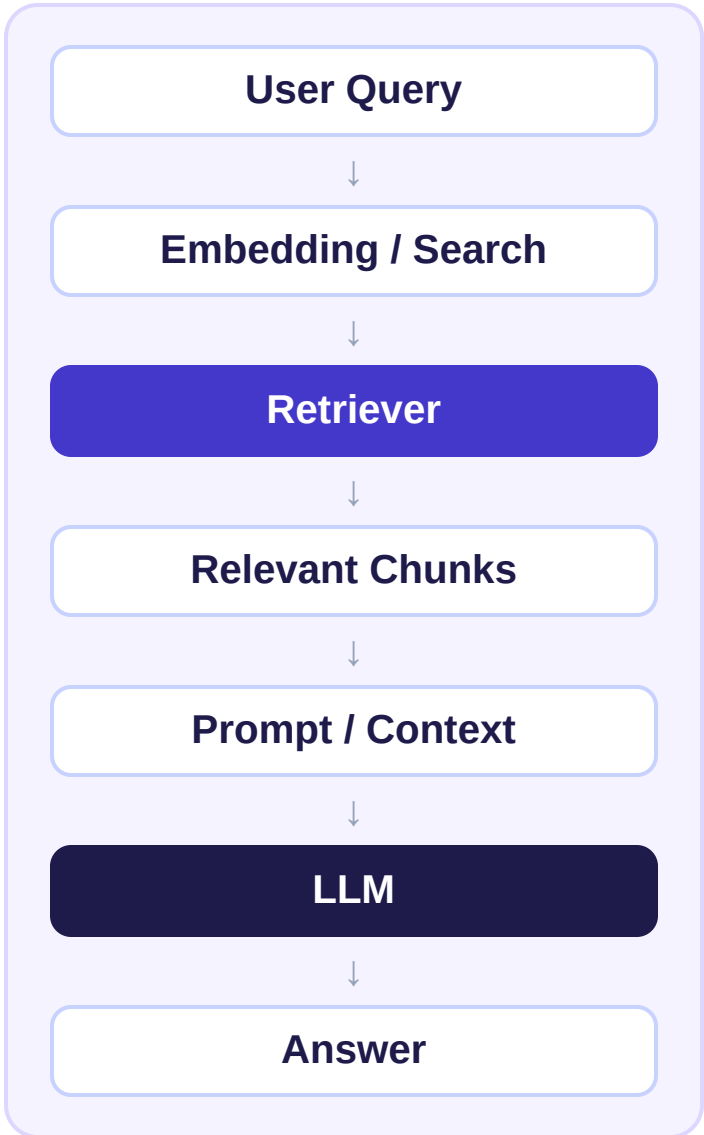
Advantages

- Simple architecture
- No retrieval pipeline required
- Preserves broad document context
- Useful for one-off analysis

Trade-offs

- More input tokens
- Potentially higher cost
- Potentially higher latency
- Still limited by the model's context capacity

RAG = Retrieve First, Generate Second



RAG does **not** put the entire knowledge base into the prompt.

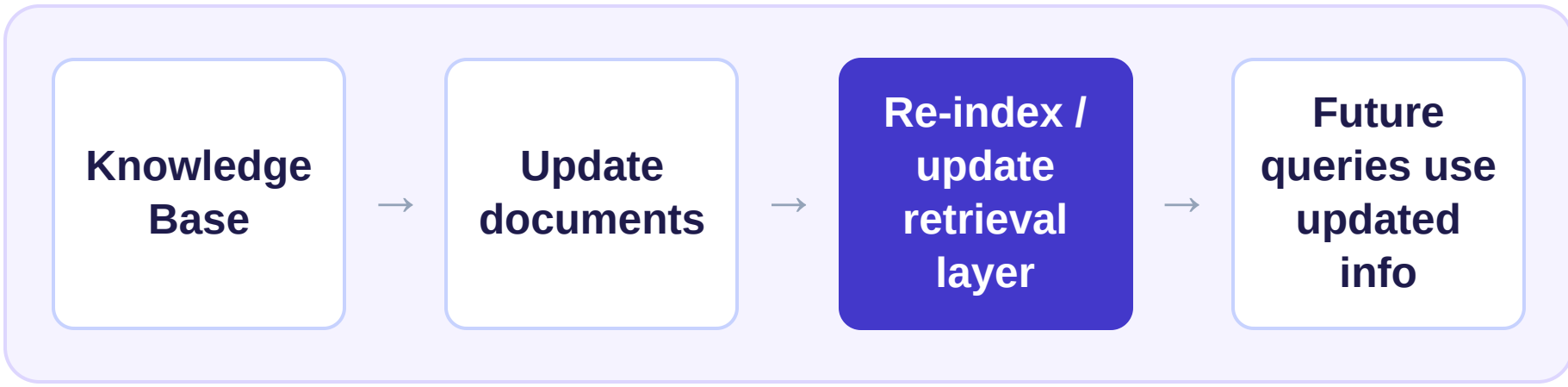
1. Store / index knowledge
2. Receive the user's query
3. Search for relevant information
4. Retrieve a smaller set of relevant content
5. Put it into the model's context
6. Generate the answer

RAG is a dynamic knowledge access pattern. It doesn't replace the context window — it selects what enters it.

When Your Knowledge Changes Frequently

HR policies · Product catalogs · Internal documentation · Support articles · Legal documents · Customer data · Inventory · Pricing · Operational databases

If the source data changes frequently, **repeatedly fine-tuning the model is usually impractical**. With RAG, you update the data — not the model:



RAG separates changing knowledge from model behavior.

Fine-Tuning Changes Model Behavior

Fine-tuning trains a pretrained model further on a **curated dataset** to adapt its behavior to a particular task or style.

Classification **Structured output** **Domain-specific syntax**

Consistent response style **Specialized instruction following**



✕ Fine-tuning does not mean

"Upload my company's latest documents and the model will always know them."

✓ Instead

It primarily changes model **behavior and patterns** — not a live knowledge database.

Fine-Tuning vs. RAG: Knowledge vs. Behavior

RAG

Designed for:

- Dynamic knowledge
- Current information
- Large external knowledge bases
- Source-grounded answers
- Updating information without retraining the model

Knowledge → Retrieve → Context → Answer

Fine-Tuning

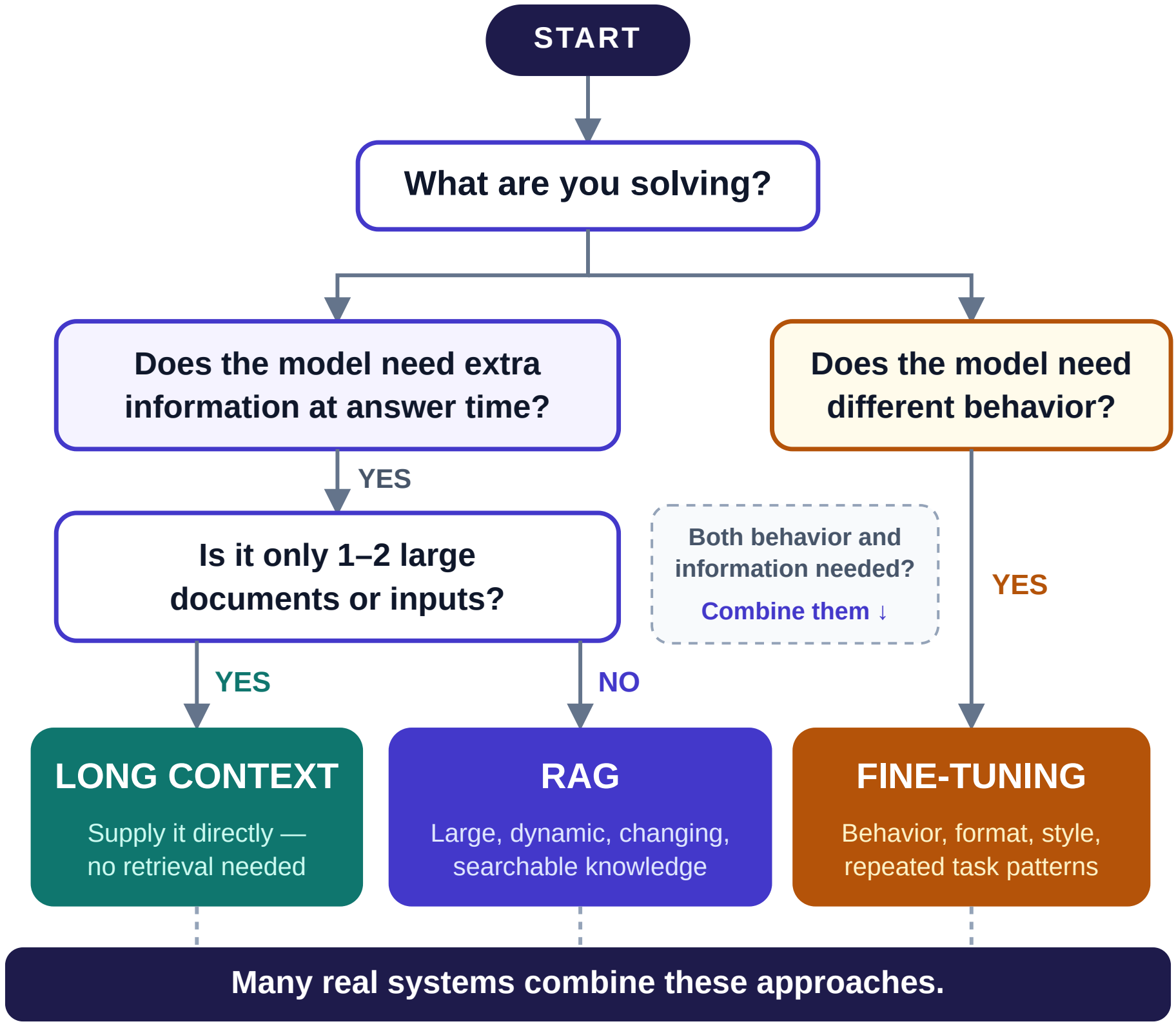
Designed for:

- Behavior adaptation
- Consistent style
- Output formatting
- Task specialization
- Domain-specific patterns

Examples → Training → Model Behavior

RAG gives the model information. Fine-tuning teaches the model a behavior.

The 3-Way Decision Tree



The Decision Matrix

Factor	Long Context	RAG	Fine-Tuning
Main purpose	Analyze supplied information	Retrieve external knowledge	Change behavior
Knowledge freshness	Immediate	Dynamic	Static until retrained
Large knowledge base	Limited	Strong fit	Poor fit
One-off documents	Strong fit	Possible	Usually unnecessary
Output style	Limited	Limited	Strong
Upfront engineering	Low	Medium / High	High
Per-query token usage	Can be high	Usually lower	Usually lower inference input
Updating knowledge	Change prompt / context	Update retrieval data	Retrain
Best for	**Large immediate inputs**	**Dynamic knowledge**	**Behavior / task adaptation**

No technique is universally superior — the best fit depends on your workload.

What Does Each Approach Cost?

Long Context

Grows with:

- Number of input tokens
- Repeated document inclusion
- Model pricing
- Latency

More tokens per request
↓
Potentially higher query cost

RAG

Includes:

- Embeddings / indexing
- Vector / database infrastructure
- Retrieval
- Reranking
- LLM generation

Retrieval infrastructure
+
Smaller context per request

Fine-Tuning

Includes:

- Dataset preparation
- Training / fine-tuning
- Evaluation
- Model hosting / inference
- Maintenance

Higher upfront investment
+
Behavior adapted into the model

No fixed dollar figures on purpose. Cost depends on the model, provider, token volume, and workload — measure your own before deciding.

Accuracy Is Not Just About the Technique

Long Context

Potential issues:

- Relevant information buried in large inputs
- Context overload
- Increased token usage
- Model-specific long-context behavior

RAG

Potential issues:

- Poor chunking
- Bad embeddings
- Weak retrieval
- Missing relevant documents
- Incorrect ranking
- Stale index

Fine-Tuning

Potential issues:

- Poor training data
- Overfitting
- Inconsistent examples
- Behavior drift
- Knowledge becoming outdated

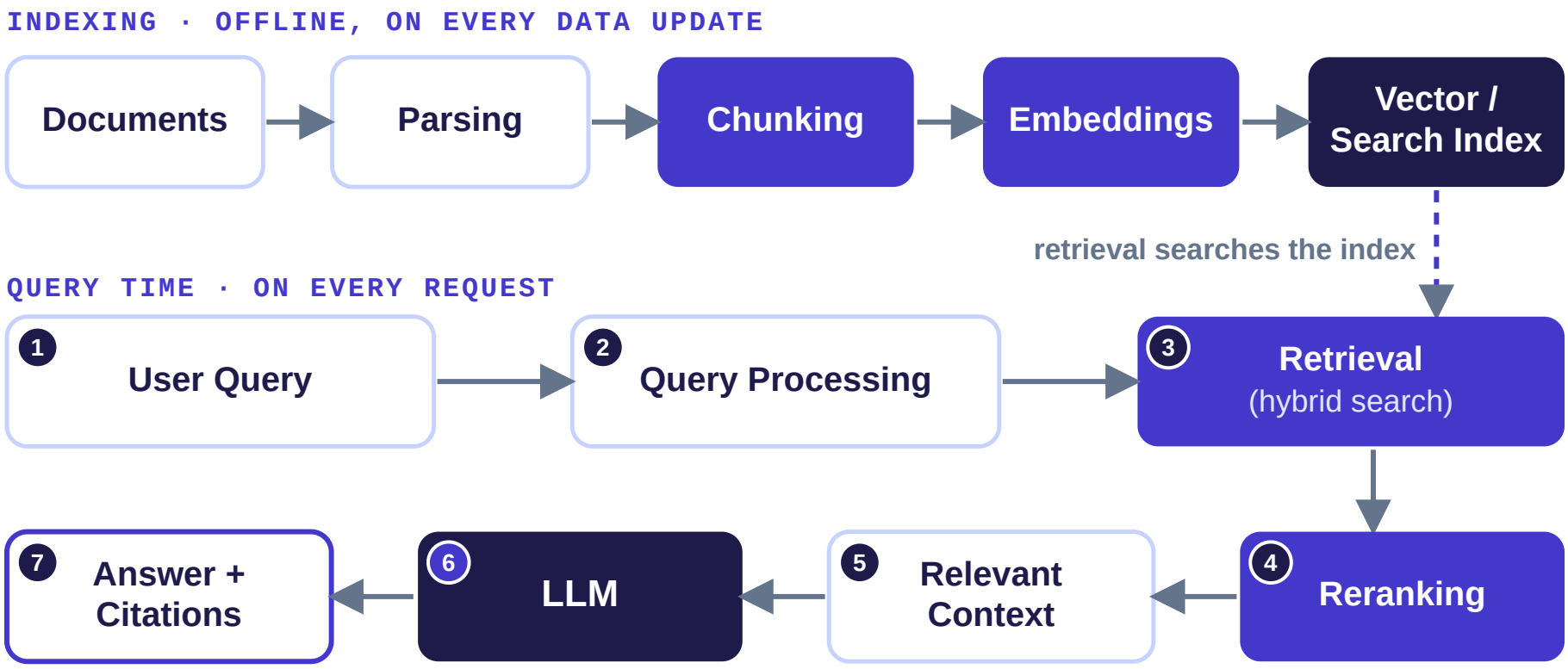
Final Answer Quality

↓

Model + **Prompt** + **Data Quality** + **Retrieval Quality** + **Evaluation**

Architecture alone does not guarantee accuracy.

Production RAG Pipeline



Chunking
Split documents into retrievable pieces

Embeddings
Encode chunks as vectors for semantic search

Hybrid search
Combine keyword and vector retrieval

Reranking
Reorder candidates by relevance

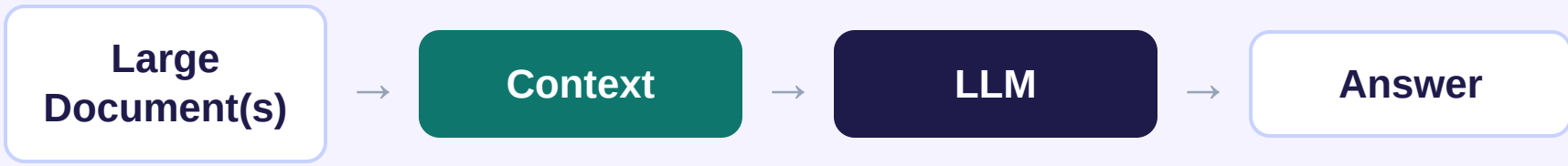
Context construction
Select and format chunks within the token budget

Grounded generation
Answer from retrieved sources, with citations

Every stage can degrade quality — evaluate retrieval separately from generation.

The Simplest Architecture

LONG CONTEXT



RAG — ADDS A RETRIEVAL LAYER

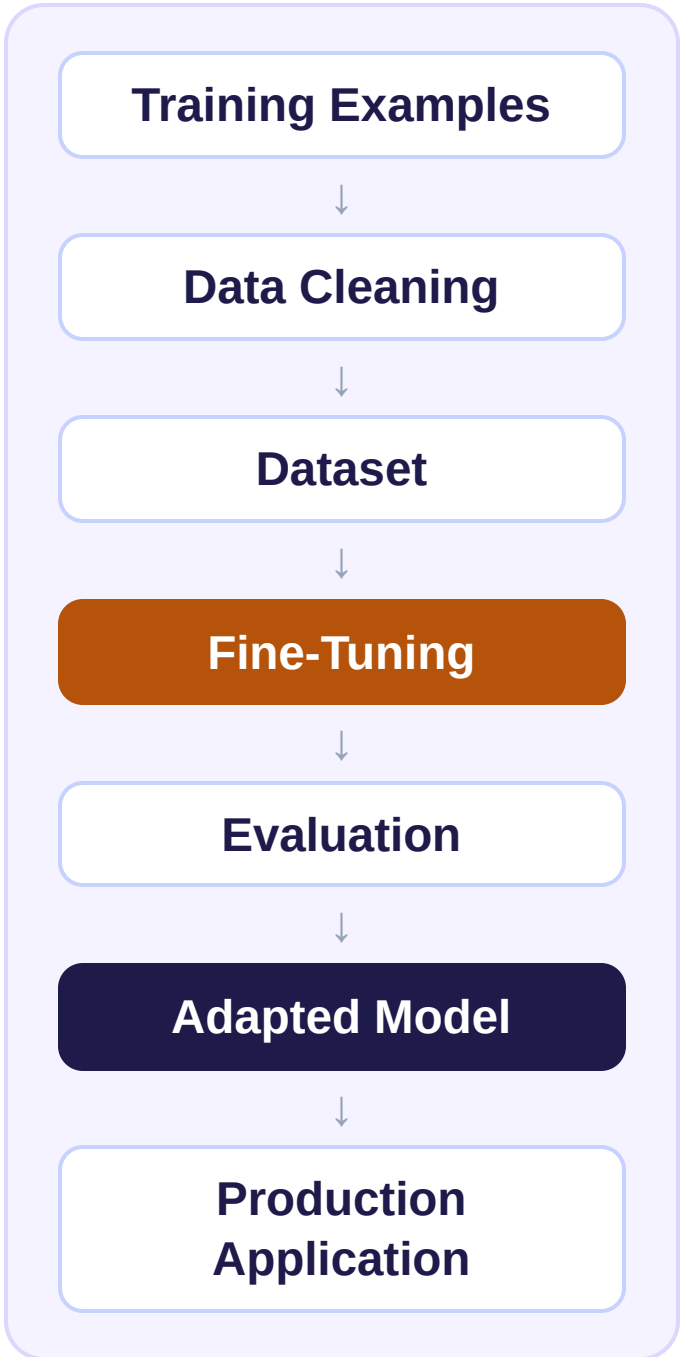


The simpler architecture may be preferable when:

- Small number of documents
- One-time analysis
- Document comparison
- No persistent retrieval infrastructure needed
- Simplicity is important

Don't build a retrieval system when simply supplying the relevant documents is sufficient.

Fine-Tuning Pipeline



EXAMPLE: TICKET TRIAGE

Input: "Customer complaint"

```
{
  "category": "billing",
  "priority": "high",
  "action":   "refund_review"
}
```

Helps the model become **more consistent** at a repeated task or output format.

If the answer depends on **today's company policy**, use a current knowledge source such as RAG — don't rely solely on training data.

Real Systems Often Use All Three

EXAMPLE: CUSTOMER-SUPPORT AI

User → AI Application

Fine-Tuned Model
Behavior
Consistent classification and output behavior.

RAG
Knowledge
Retrieves current product and policy information.

Long Context
Immediate input
Analyzes the customer's long conversation or attached document.

LLM
a fine-tuned model reading retrieved + supplied context

Complementary techniques — not mutually exclusive technologies.

Which One Fits the Scenario?

SCENARIO 1

"Summarize this 150-page contract."

→ **Long Context may be appropriate**

SCENARIO 2

"Answer questions over 500,000 constantly changing company documents."

→ **RAG is a natural architecture to consider**

SCENARIO 3

"Always return support tickets in our exact JSON structure."

→ **Fine-Tuning may help**

→ **+ strong prompting, schema validation**

SCENARIO 4

"Analyze today's policy document and classify the customer's request."

→ **RAG**

→ **+ behavior adaptation may help**

SCENARIO 5

"Compare two large technical specifications."

→ **Long Context may be appropriate**

Ask These 7 Questions Before Choosing

1	Is the information changing frequently?	→ Consider RAG
2	Is the information already in one or two large documents?	→ Consider Long Context
3	Do you need the model to behave differently?	→ Consider Fine-Tuning
4	Does the model need access to a large knowledge base?	→ Consider RAG
5	Is this a repeated task with consistent input / output patterns?	→ Consider Fine-Tuning
6	Can the relevant information fit comfortably into the context?	→ Long Context may suffice
7	Can multiple approaches solve the problem?	→ Consider a hybrid

Start with the simplest architecture that satisfies your requirements.

Final Decision Matrix: What Do You Need?

More input right now?

↓

LONG CONTEXT

"Give the model more information now."

BEST SUITED FOR

- Large one-off inputs
- Document analysis
- Document comparison

Dynamic knowledge?

↓

RAG

"Find the right information when needed."

BEST SUITED FOR

- Dynamic knowledge
- Enterprise search
- Frequently changing data

Different behavior?

↓

FINE-TUNING

"Teach the model a more specialized behavior."

BEST SUITED FOR

- Consistent style
- Output formats
- Task specialization
- Domain-specific patterns

Context gives the model information.
RAG retrieves the right information.
Fine-tuning changes how the model behaves.

In production, the answer is often a combination of all three.